



Capítulo 1

Arquitecturas web

1.1. Tipos de aplicaciones

Las aplicaciones se pueden clasificar de varias formas:

1. Por **dónde** y **cómo** se ejecutan

- Aplicaciones de **escritorio**: Se instalan en una computadora.
- Aplicaciones **móviles** (nativas): Se instalan en teléfonos móviles (Android, iOS).
- **Aplicaciones web: Se usan desde un navegador, sin instalación.**
- Aplicaciones **híbridas**: Mezclan web y móvil.
- **PWA** (Progressive Web Apps): Son aplicaciones web que se comportan como apps móviles (pueden funcionar offline, enviarte notificaciones y añadirse al escritorio).

2. Según su **propósito**

- **Productividad**: Google Docs, Trello, Notion.
- **Comunicación**: WhatsApp, Slack, Zoom.
- **Entretenimiento**: Netflix, Spotify, videojuegos.
- **Comercio electrónico**: Amazon, Mercado Libre.
- **Educación**: Duolingo, Moodle, Khan Academy.
- **Finanzas**: PayPal, apps de bancos.

3. Según su arquitectura:

- **Monolíticas:** todo en un solo bloque.
- **Cliente-servidor:** separan frontend y backend.
- **Microservicios:** divididas en pequeños módulos independientes.
- **Serverless** ejecutan funciones bajo demanda en la nube.



Figura 1.1: Tipos de aplicaciones

1.2. Aplicaciones web

Una aplicación web es un software al que se accede y utiliza a través de un **navegador web** (Chrome, Firefox, Edge, Safari, ...), sin necesidad de instalarlo. En lugar de descargarse como un programa tradicional, la aplicación web funciona en un servidor remoto y el usuario interactúa con ella mediante Internet.

Las principales características de una aplicación web son:

- Acceso desde el navegador, no requiere instalación.
- Multiplataforma, funciona en PC, tablet o móvil.
- Actualizaciones automáticas, los cambios se hacen en el servidor, no en cada dispositivo.

Un sitio o página web suele ser informativo (ej: página de un restaurante con menú y ubicación).

Una aplicación web es **interactiva**, permite al usuario realizar acciones (registrarse, comprar, editar documentos, chatear, etc.).

1.3. ¿Qué es una arquitectura web?

Una arquitectura web es **la forma en que se estructuran y organizan los distintos componentes de una aplicación web**, tanto en sus componentes visibles (lo que ve el usuario) como en los procesos internos (lo que ocurre en el servidor).

Describe cómo se comunican y funcionan entre sí las distintas partes de un sistema web para que el usuario pueda interactuar con él desde un navegador.

Los principales **componentes** de una arquitectura web:

- **Cliente (Front-end)**

- * Es la parte visible para el usuario.
- * Incluye la interfaz gráfica hecha con tecnologías como HTML, CSS y JavaScript.
- * Se ejecuta en el navegador del usuario.

- **Servidor (Back-end)**

- * Procesa las solicitudes que llegan desde el cliente.
- * Se encarga de la lógica de negocio, autenticación, seguridad, etc.
- * Puede estar hecho con lenguajes como Node.js, Java, Python, PHP, .NET, etc...

- **Base de datos**

- * Donde se almacenan los datos que usa la aplicación (usuarios, productos, contenido, etc.).

- **Comunicación o red**

- * Normalmente se realiza a través de HTTP/HTTPS.
- * Hoy en día muchas aplicaciones usan APIs REST o GraphQL para enviar y recibir datos.

No hay que confundir **arquitectura con tecnología**. La arquitectura web es el **diseño o la forma** en que se organizan y comunican los distintos componentes de una aplicación web. Establece como fluye la información y cómo interactúan cliente, servidor y bases de datos.

Las **tecnologías** web son las **herramientas**, lenguajes, frameworks o protocolos que se usan para construir la aplicación siguiendo una arquitectura.



GraphQL es una tecnología moderna para APIs que permite consultas flexibles y precisas en un solo endpoint (url), pensada para optimizar la comunicación entre frontend y backend, especialmente en SPAs y apps móviles.



Un framework es una estructura predefinida y un conjunto de herramientas, librerías y componentes estandarizados que proporcionan una base para desarrollar aplicaciones.

Ejemplos de tecnologías web:

- * Lenguajes: HTML, CSS, JavaScript, Python, PHP, Java, ...
- * Frameworks: Django, Spring Boot, Express.js, Angular, React, Vue, Laravel.
- * Bases de datos: MySQL, PostgreSQL, MongoDB.
- * Protocolos y estándares: HTTP/HTTPS, WebSockets, REST, GraphQL.
- * Servidores: Apache, Nginx, Node.js.



*Arquitectura
web = el plano
o diseño de la
casa. Tecno-
logías web = los
materiales y he-
rramientas con
que construyes
la casa.*

1.4. Tipos de arquitecturas

1.4.1. Arquitectura monolítica

Una aplicación monolítica es aquella en la que **todo está dentro de un único proyecto** (la interfaz de usuario, la lógica de negocio y el acceso a la base de datos). Todo se desarrolla, despliega y ejecuta como una sola unidad.

Ejemplo una tienda en línea monolítica:

El usuario entra a la web y ve el catálogo de productos generados por el servidor.

El servidor gestiona todo, es decir, mostrar los productos, procesar el carrito de compras, validar el pago, guardar pedidos en la base de datos,....

Una sola aplicación contiene el frontend, el backend y la conexión a la base de datos.

```
/mi_tienda
├─ index.php      # Página principal (frontend + backend)
├─ style.css      # Estilos CSS
└─ script.js      # JS para interacción
```

Figura 1.2: Directorio arquitectura monolítica.

Código para esta tienda con estructura monolítica:

```
1
2 // Sql para la creación de la base de datos
3
4 CREATE DATABASE tienda;
5 USE tienda;
6
7 CREATE TABLE productos (
8   id INT AUTO_INCREMENT PRIMARY KEY,
9   nombre VARCHAR(100) NOT NULL,
10  precio DECIMAL(10,2) NOT NULL
11 );
12
13 INSERT INTO productos (nombre, precio) VALUES
14 ('Portatil', 1200.00),
15 ('Teclado', 25.00),
16 ('Mouse', 15.00);
```

```
1
2 // index.php
3
4 <?php
5 // Conexión a la base de datos
6 $conexion = new mysqli("localhost", "root", "usbw", "tienda");
7 if ($conexion->connect_error) {
8     die("Error de conexión: " . $conexion->connect_error);
9 }
10
11 // Consultar productos
12 $resultado = $conexion->query("SELECT * FROM productos");
13 ?>
14 <!DOCTYPE html>
15 <html lang="es">
16 <head>
17     <meta charset="UTF-8">
18     <title>Mi Tienda Monolítica</title>
19     <link rel="stylesheet" href="style.css">
20 </head>
21 <body>
22     <h1>Bienvenido a Mi Tienda</h1>
23     <div id="productos">
24         <?php while ($fila = $resultado->fetch_assoc()) { ?>
25             <div class="producto">
26                 <h2><?= $fila['nombre'] ?></h2>
27                 <p>Precio: <?= $fila['precio'] ?></p>
28                 <button onclick="agregarCarrito('<?= $fila['nombre']
29                     ?>')">Agregar al carrito</button>
30             </div>
31         <?php } ?>
32     </div>
33     <h2>Carrito:</h2>
34     <ul id="carrito"></ul>
35
36     <script src="script.js"></script>
37 </body>
38 </html>
```

```
1
2 //style.css
3
4 body {
5     font-family: Arial, sans-serif;
6     margin: 20px;
7     background: #f4f4f4;
8 }
9
10 h1 {
11     color: #333;
12 }
13
14 .producto {
15     border: 1px solid #ccc;
16     background: #fff;
17     padding: 10px;
18     margin: 10px;
```

```
19     display: inline-block;
20     width: 200px;
21     text-align: center;
22 }
23
24 button {
25     background: #28a745;
26     color: white;
27     border: none;
28     padding: 8px;
29     cursor: pointer;
30 }
31 button:hover {
32     background: #218838;
33 }
```

```
1
2 // script.js
3
4 function agregarCarrito(nombre) {
5     let carrito = document.getElementById("carrito");
6     let item = document.createElement("li");
7     item.textContent = nombre;
8     carrito.appendChild(item);
9 }
```

Ventajas:

Fácil de iniciar y desplegar.

Menor complejidad al comienzo.

Buena opción para proyectos pequeños o medianos.

Desventajas:

Difícil de escalar cuando crece mucho.

Si una parte falla, puede afectar a toda la app.

Los cambios grandes requieren volver a desplegar todo el sistema.



Colocar la carpeta mi-tienda en el directorio root del usbwebserver y ver desde localhost

1.4.2. Arquitectura cliente-servidor clásica

En la arquitectura cliente-servidor clásica el cliente (navegador) solicita datos al servidor. El servidor responde normalmente en JSON o XML, y el cliente se encarga de mostrarlo.

Es más separado que el monolítico, porque el **frontend no está mezclado con el backend**.

```
/mi_tienda_api
├ cliente.html      # Frontend (HTML, CSS, JS)
├ style.css         # Estilos
├ script.js         # Lógica cliente
└ api.php           # Backend PHP (API REST simple)
```

Figura 1.3: Directorio arquitectura cliente servidor.

```
1
2 // api.php
3
4 <?php
5 header("Content-Type: application/json");
6
7 // Conexión MySQL
8 $conexion = new mysqli("localhost", "root", "usbw", "tienda");
9 if ($conexion->connect_error) {
10     die(json_encode(["error" => "Error de conexión"]));
11 }
12
13 // Consultar productos
14 $resultado = $conexion->query("SELECT * FROM productos");
15 $productos = [];
16
17 while ($fila = $resultado->fetch_assoc()) {
18     $productos[] = $fila;
19 }
20
21 echo json_encode($productos);
22 ?>
```

```
1
2 // cliente.html (Frontend)
3
4 // El cliente pide datos con fetch al servidor
5
6 <!DOCTYPE html>
7 <html lang="es">
```

```
8 <head>
9   <meta charset="UTF-8">
10  <title>Tienda Cliente-Servidor</title>
11  <link rel="stylesheet" href="style.css">
12 </head>
13 <body>
14   <h1>Tienda con Arquitectura Cliente-Servidor</h1>
15   <div id="productos"></div>
16
17   <h2>Carrito:</h2>
18   <ul id="carrito"></ul>
19
20   <script src="script.js"></script>
21 </body>
22 </html>
```

```
1
2 // script.js (Lógica cliente)
3
4 async function cargarProductos() {
5   let respuesta = await fetch("api.php");
6   let productos = await respuesta.json();
7
8   let contenedor = document.getElementById("productos");
9   productos.forEach(p => {
10     let div = document.createElement("div");
11     div.className = "producto";
12     div.innerHTML = `
13       <h2>${p.nombre}</h2>
14       <p>Precio: ${p.precio}</p>
15       <button onclick="agregarCarrito('${p.nombre}')">Agregar
16         al carrito</button>
17     `;
18     contenedor.appendChild(div);
19   });
20
21 function agregarCarrito(nombre) {
22   let carrito = document.getElementById("carrito");
23   let item = document.createElement("li");
24   item.textContent = nombre;
25   carrito.appendChild(item);
26 }
27
28 cargarProductos();
```

En el monolítico, el PHP genera el HTML directamente (todo junto).

En el cliente-servidor, el PHP solo expone datos (JSON) y el navegador los pinta con JS.

Ventajas:

Separación clara de responsabilidades: cliente maneja la interfaz, servidor maneja lógica y datos.

Mejor escalabilidad: se puede mejorar el servidor (más potencia, balanceo de carga) sin tocar el cliente.

Seguridad: datos y lógica críticos permanecen en el servidor, no expuestos al cliente.

Facilita múltiples clientes: distintas aplicaciones (web, escritorio, móvil) pueden conectarse al mismo servidor.

Desventajas:

Dependencia del servidor: si este cae, todos los clientes quedan inoperativos.

Latencia: cada operación requiere comunicación con el servidor, lo que puede hacer más lenta la experiencia.

Complejidad mayor que un monolito pequeño: requiere gestionar protocolos de comunicación, sesiones, etc.

Costos más altos en infraestructura (mantenimiento de servidores, redes, balanceadores).

1.4.3. JSON (JavaScript Object Notation)

```
1
2 {
3   "arrayColores":[{
4     "nombreColor":"rojo",
5     "valorHexadec":"#f00"
6   },
7   {
8     "nombreColor":"verde",
9     "valorHexadec":"#0f0"
10  },
11  {
12    "nombreColor":"azul",
13    "valorHexadec":"#00f"
14  },
15  {
16    "nombreColor":"cyan",
17    "valorHexadec":"#0ff"
18  },
19  {
20    "nombreColor":"magenta",
21    "valorHexadec":"#f0f"
22  },
23  {
24    "nombreColor":"amarillo",
25    "valorHexadec":"#ff0"
26  },
27  {
28    "nombreColor":"negro",
29    "valorHexadec":"#000"
30  }
31 ]
32 }
```

<https://www.youtube.com/watch?v=9he40ewlYFo&t=212s>(9:38)

En el mundo de la programación, el intercambio de datos es fundamental y uno de los formatos más utilizados hoy en día para ello es JSON.

JSON, cuyo nombre corresponde a las siglas JavaScript Object Notation o Notación de Objetos de JavaScript, es un formato ligero de intercambio de datos, que resulta sencillo de leer y escribir para los programadores y simple de interpretar y generar para las máquinas.

JSON es un formato de texto completamente independiente de lenguaje. Son muchos los lenguajes de programación que proporcionan métodos para analizar una cadena de texto con este formato en un objeto nativo y viceversa.

JSON permite usar seis tipos de valores: nulo, números, cadenas, booleanos, matrices y objetos.

Características de JSON:

- * JSON es solo un formato de datos.
- * Requiere usar comillas dobles para las cadenas y los nombres de propiedades. Las comillas simples no son válidas.
- * Una coma o dos puntos mal ubicados pueden producir que un archivo JSON no funcione.
- * Puede tomar la forma de cualquier tipo de datos que sea válido para ser incluido en un JSON, no solo arreglos u objetos. Así, por ejemplo, una cadena o un número único podrían ser objetos JSON válidos.
- * A diferencia del código JavaScript, en el que las propiedades del objeto pueden no estar entre comillas, en JSON solo las cadenas entre comillas pueden ser utilizadas como propiedades.

<https://www.json.org/json-es.html>

Ejemplos:

```
1
2 let objetoJSON = {"usuario":"user",
3   "password":"123456"};
4 alert(objetoJSON.usuario); // user
5
6 let objetos = {"Personas":[
7   {"nombre":"Juan","edad":23},
8   {"nombre":"Carlos","edad":17}
9   ]};
10
11 alert(objetos.Personas[1].nombre); // Carlos
```

En javascript, para serializar utilizamos el método **stringify** y para deserializar el **parse**.

json.stringify convierte un valor dado en javascript a una cadena JSON.

https://developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Objetos_globales/JSON/stringify

json.parse analiza una cadena de texto como JSON, transformando opcionalmente el valor producido por el análisis.

https://developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Objetos_globales/JSON/parse

<https://frontendlabs.io/1490--json-que-es-json-parse-json-stringify>

```
1
2 // Objeto js
3
4 let biblioteca = {
5   autores: [
6     { nombre: 'Miguel de Cervantes',
```

```
7             nacimiento: new Date(1547, 8, 29),
8             lugar: 'Alcalá de Henares, Madrid, España'
9         },
10        {     nombre: 'William Shakespeare',
11             bautizo: new Date(1564, 3, 26),
12             lugar: 'Stratford-upon-Avon, Warwickshire, Reino
13                 Unido'
14         }
15    ]
16
17
18    alert( typeof biblioteca + "\n" + biblioteca ); // object
19
20    let biblioString = JSON.stringify(biblioteca);
21    alert( typeof biblioString + "\n" + biblioString); // String
22
23    let biblioNuevo = JSON.parse(biblioString);
24    alert(typeof biblioNuevo + "\n" + biblioNuevo); // Object
```

Validador online: <https://jsonlint.com/>

1.4.4. Arquitectura en tres capas

La aplicación se organiza en tres niveles separados:

1. Capa de **Presentación**: lo que ve el usuario (HTML, CSS, JS).
2. Capa de **Lógica de Negocio**: reglas y procesos de la aplicación (PHP que procesa las peticiones).
3. Capa de **Datos**: la base de datos (MySQL).

```
/mi_tienda_3capas
├─ presentacion
│   ├─ index.html      # Frontend
│   ├─ style.css
│   └─ script.js
├─ negocio
│   └─ productos.php   # Lógica de negocio (API)
└─ datos
    └─ db.php           # Conexión a la base de datos
```

Figura 1.4: Directorio arquitectura tres capas.

```
1
2 //datos/db.php
3
4 <?php
5 class Database {
6     private $host = "localhost";
7     private $user = "root";
8     private $pass = "usbw";
9     private $db   = "tienda";
10    public $conexion;
11
12    public function __construct() {
13        $this->conexion = new mysqli($this->host, $this->user,
14                                     $this->pass, $this->db);
15        if ($this->conexion->connect_error) {
16            die("Error de conexión: " .
17                $this->conexion->connect_error);
18        }
19    }
20 }
```

```
1
2 //negocio/productos.php
3
4 <?php
5 header("Content-Type: application/json");
6 require_once("../datos/db.php");
7
8 $db = new Database();
9 $sql = "SELECT * FROM productos";
10 $resultado = $db->conexion->query($sql);
11
12 $productos = [];
13 while ($fila = $resultado->fetch_assoc()) {
14     $productos[] = $fila;
15 }
16
17 echo json_encode($productos);
18 ?>
```

```
1
2 // presentacion/index.html
3
4 // La capa de presentación consume la API de la capa de negocio.
5
6 <!DOCTYPE html>
7 <html lang="es">
8 <head>
9     <meta charset="UTF-8">
10     <title>Tienda 3 Capas</title>
11     <link rel="stylesheet" href="style.css">
12 </head>
13 <body>
14     <h1>Tienda con Arquitectura en 3 Capas</h1>
15     <div id="productos"></div>
16
17     <h2>Carrito:</h2>
18     <ul id="carrito"></ul>
19
20     <script src="script.js"></script>
21 </body>
22 </html>
```

```
1
2 // presentacion/script.js
3
4 // El frontend pide datos a la capa de negocio
5     ('../negocio/productos.php'):
6
7 async function cargarProductos() {
8     let respuesta = await fetch("../negocio/productos.php");
9     let productos = await respuesta.json();
10
11     let contenedor = document.getElementById("productos");
12     productos.forEach(p => {
13         let div = document.createElement("div");
14         div.className = "producto";
15         div.innerHTML = '
```



```
15         <h2>${p.nombre}</h2>
16         <p>Precio: ${p.precio}</p>
17         <button onclick="agregarCarrito('${p.nombre}')">Agregar
           al carrito</button>
18     '
19     contenedor.appendChild(div);
20 }
21 }
22
23 function agregarCarrito(nombre) {
24     let carrito = document.getElementById("carrito");
25     let item = document.createElement("li");
26     item.textContent = nombre;
27     carrito.appendChild(item);
28 }
29
30 cargarProductos();
```

Diferencias clave:

- Cliente-Servidor: frontend pide datos a backend, pero la lógica y datos están mezclados en el servidor.
- Tres capas: la lógica y la base de datos están separadas claramente en capas distintas.

Ventajas:

Separación clara de responsabilidades. Cada capa tiene un rol definido (UI, lógica, datos), lo que mejora la organización del código.

Mantenibilidad: Cambiar la interfaz o la base de datos no obliga a reescribir toda la aplicación. Ejemplo: puedes cambiar de MySQL a PostgreSQL sin tocar la lógica de negocio ni la UI.

Reutilización: La capa de negocio puede ser usada por distintos clientes (web, móvil, escritorio).

Escalabilidad: Puedes escalar cada capa por separado (más servidores de BD, más servidores de aplicación).

Seguridad: Los clientes nunca acceden directamente a la base de datos, solo a la lógica de negocio, lo que reduce riesgos.

Desventajas:

Mayor complejidad inicial. Se necesita diseñar e implementar las tres capas correctamente, lo que puede ser más costoso que un monolito.

Rendimiento más bajo: Las llamadas entre capas generan más latencia que en un monolito.

Sobrecoste en proyectos pequeños. Para aplicaciones simples, usar tres capas puede ser innecesario y ralentizar el desarrollo.

Curva de aprendizaje: Requiere que el equipo entienda conceptos de modularización, capas y buenas prácticas.

1.4.5. Microservicios

En 3 capas, todo sigue siendo un solo proyecto con divisiones internas.

En microservicios, separamos la aplicación en múltiples servicios independientes. Cada servicio tiene su propia responsabilidad y se comunica por API REST.

En nuestro ejemplo, cada servicio será un módulo PHP independiente, y el frontend (HTML+JS) los consultará.

```
/mi_tienda_microservicios
├── frontend
│   ├── index.html
│   ├── style.css
│   └── script.js
├── servicio_productos
│   ├── db.php
│   └── productos.php
└── servicio_carrito
    └── carrito.php
```

Figura 1.5: Directorio arquitectura microservicios.

```
1
2 // servicio_productos/db.php
3
4 // Conexión al MySQL (solo para este servicio):
5
6 <?php
```

```

7 class Database {
8     private $host = "localhost";
9     private $user = "root";
10    private $pass = "usbw";
11    private $db = "tienda";
12    public $conexion;
13
14    public function __construct() {
15        $this->conexion = new mysqli($this->host, $this->user,
16            $this->pass, $this->db);
17        if ($this->conexion->connect_error) {
18            die("Error de conexión: " .
19                $this->conexion->connect_error);
20        }
21    }
22 }
23 ?>

```

```

1
2 // servicio_productos/productos.php
3
4 // Microservicio de productos:
5
6 <?php
7 header("Content-Type: application/json");
8 require_once("db.php");
9
10 $db = new Database();
11 $sql = "SELECT * FROM productos";
12 $resultado = $db->conexion->query($sql);
13
14 $productos = [];
15 while ($fila = $resultado->fetch_assoc()) {
16     $productos[] = $fila;
17 }
18
19 echo json_encode($productos);
20 ?>

```

```

1
2 // servicio_carrito/carrito.php
3
4 // Microservicio de carrito (simula almacenamiento en sesión):
5
6 <?php
7 session_start();
8 header("Content-Type: application/json");
9
10 if (!isset($_SESSION["carrito"])) {
11     $_SESSION["carrito"] = [];
12 }
13
14 if ($_SERVER["REQUEST_METHOD"] === "POST") {
15     $input = json_decode(file_get_contents("php://input"), true);
16     if (isset($input["producto"])) {
17         $_SESSION["carrito"][] = $input["producto"];
18     }
19 }

```



Una sesión es un mecanismo que permite a un servidor recordar información sobre un usuario mientras navega por un sitio web. Como HTTP es un protocolo sin estado (stateless), cada petición del navegador al servidor es independiente y no recuerda nada de la anterior. La sesión soluciona esto, permitiendo que la aplicación mantenga un estado para cada usuario mientras dure su visita.

```
20
21 echo json_encode($_SESSION["carrito"]);
22 ?>
```

```
1
2 // frontend/index.html
3
4 // Frontend que consume los microservicios:
5
6 <!DOCTYPE html>
7 <html lang="es">
8 <head>
9   <meta charset="UTF-8">
10  <title>Tienda Microservicios</title>
11  <link rel="stylesheet" href="style.css">
12 </head>
13 <body>
14  <h1>Tienda con Arquitectura de Microservicios</h1>
15  <div id="productos"></div>
16
17  <h2>Carrito:</h2>
18  <ul id="carrito"></ul>
19
20  <script src="script.js"></script>
21 </body>
22 </html>
```

```
1
2 // frontend/script.js
3
4 // Aquí el frontend llama a dos microservicios: productos y carrito.
5
6 async function cargarProductos() {
7   let respuesta = await
8     fetch("../servicio_productos/productos.php");
9   let productos = await respuesta.json();
10
11   let contenedor = document.getElementById("productos");
12   productos.forEach(p => {
13     let div = document.createElement("div");
14     div.className = "producto";
15     div.innerHTML = `
16       <h2>${p.nombre}</h2>
17       <p>Precio: ${p.precio}</p>
18       <button onclick="agregarCarrito('${p.nombre}')">Agregar
19         al carrito</button>
20     `;
21     contenedor.appendChild(div);
22   });
23 }
24
25 async function agregarCarrito(nombre) {
26   let respuesta = await fetch("../servicio_carrito/carrito.php", {
27     method: "POST",
28     headers: {"Content-Type": "application/json"},
29     body: JSON.stringify({ producto: nombre })
30   });
31 }
```

```
30     let carrito = await respuesta.json();
31     let lista = document.getElementById("carrito");
32     lista.innerHTML = "";
33     carrito.forEach(item => {
34         let li = document.createElement("li");
35         li.textContent = item;
36         lista.appendChild(li);
37     });
38 }
39
40 cargarProductos();
```

Ventajas:

Escalabilidad independiente: Cada microservicio se puede escalar de forma separada según la carga (ej: solo el de pagos).

Desarrollo paralelo: Equipos distintos pueden trabajar en diferentes microservicios sin pisarse entre ellos.

Flexibilidad tecnológica: Cada microservicio puede estar desarrollado con el lenguaje o framework más adecuado (ej: Python para IA, Node.js para tiempo real).

Mantenibilidad y despliegue rápido: Se actualiza un servicio sin necesidad de redeployar todo el sistema.

Resiliencia: Si un microservicio falla (ej: notificaciones), el resto del sistema puede seguir funcionando.

Desventajas:

Complejidad de arquitectura: Requiere orquestar múltiples servicios, comunicación entre ellos y gestión de dependencias.

Sobrecoste en comunicación: La interacción entre microservicios genera más latencia que una arquitectura monolítica.

Gestión de datos distribuida: Cada microservicio suele tener su propia base de datos por lo que es más difícil asegurar consistencia.

Infraestructura más costosa: Necesita contenedores, balanceadores, monitoreo, logs centralizados, etc.

Curva de aprendizaje: El equipo debe dominar patrones como API Gateway, descubrimiento de servicios, tolerancia a fallos.

Debugging complicado: Rastrear un error entre varios microservicios puede ser mucho más difícil que en un monolito.

1.4.6. API REST

API = Application Programming Interface. Es una interfaz que permite que dos aplicaciones se comuniquen entre sí.

REST = Representational State Transfer. Es un estilo de arquitectura para diseñar APIs que usan el protocolo HTTP de forma sencilla y estándar.

Una API REST es un conjunto de endpoints HTTP (URLs) que permiten a un cliente (por ejemplo, una app web o móvil) consultar o modificar datos en un servidor.

La comunicación vía HTTP usa métodos estándar:

- * GET: obtener datos
- * POST: insertar un recurso
- * PUT: actualizar un recurso
- * DELETE: eliminar un recurso

Recursos representados por URLs

- * GET /productos - lista de productos

```
1
2     // lo que devolverá get/productos
3     [
4         { "id": 1, "nombre": "Portátil", "precio": 1200 },
5         { "id": 2, "nombre": "Mouse", "precio": 20 }
6     ]
```

- * GET /productos/5 - producto con ID 5

Ventajas de usar API REST:

Simplicidad: se entiende fácil, porque usa HTTP estándar.

Multiplataforma: la misma API puede ser consumida por apps web, móviles o de escritorio.

Escalabilidad: ideal para arquitecturas modernas (3 capas, microservicios, serverless).

1.4.7. ServerLess

En una arquitectura Serverless, en lugar de tener un servidor siempre encendido, se usan **funciones bajo demanda en la nube** (ej. AWS Lambda, Azure Functions, Google Cloud Functions).

El frontend hace peticiones HTTP a esas funciones, y ellas responden con los datos.

La infraestructura (escalado, disponibilidad, etc.) la gestiona el proveedor cloud.

Como no podemos levantar AWS Lambda, en el ejemplo, vamos a realizar una simulación con php, como si fueran funciones serverless.

```
/mi_tienda_serverless
├── frontend
│   ├── index.html
│   ├── style.css
│   └── script.js
└── functions
    ├── getProductos.php    # "Función serverless"
    └── addCarrito.php      # Otra "función serverless"
```

Figura 1.6: Directorio arquitectura serverless (simulación).

```
1
2 // functions/getProductos.php
3
4 // Función que devuelve productos (simulando AWS Lambda):
5
6 <?php
7 header("Content-Type: application/json");
8
9 $conexion = new mysqli("localhost", "root", "usbw", "tienda");
10 if ($conexion->connect_error) {
11     http_response_code(500);
12     echo json_encode(["error" => "Error en la conexión"]);
13     exit;
14 }
15
16 $resultado = $conexion->query("SELECT * FROM productos");
17 $productos = [];
18
19 while ($fila = $resultado->fetch_assoc()) {
```



```
20     $productos[] = $fila;
21 }
22
23 echo json_encode($productos);
24 ?>
```

```
1
2 // functions/addCarrito.php'
3
4 // Función que gestiona el carrito (almacenado en sesión):
5
6 <?php
7 session_start();
8 header("Content-Type: application/json");
9
10 if (!isset($_SESSION["carrito"])) {
11     $_SESSION["carrito"] = [];
12 }
13
14 if ($_SERVER["REQUEST_METHOD"] === "POST") {
15     $input = json_decode(file_get_contents("php://input"), true);
16     if (isset($input["producto"])) {
17         $_SESSION["carrito"][] = $input["producto"];
18     }
19 }
20
21 echo json_encode($_SESSION["carrito"]);
22 ?>
```

```
1
2 // frontend/index.html'
3
4 // Interfaz web que llama a las funciones serverless:
5
6 <!DOCTYPE html>
7 <html lang="es">
8 <head>
9     <meta charset="UTF-8">
10    <title>Tienda Serverless</title>
11    <link rel="stylesheet" href="style.css">
12 </head>
13 <body>
14    <h1>Tienda con Arquitectura Serverless</h1>
15    <div id="productos"></div>
16
17    <h2>Carrito:</h2>
18    <ul id="carrito"></ul>
19
20    <script src="script.js"></script>
21 </body>
22 </html>
```

```
1
2 //frontend/script.js'
3
4 // El frontend ahora consume directamente las funciones:
5
```

```
6  async function cargarProductos() {
7      let respuesta = await fetch("../functions/getProductos.php");
8      let productos = await respuesta.json();
9
10     let contenedor = document.getElementById("productos");
11     productos.forEach(p => {
12         let div = document.createElement("div");
13         div.className = "producto";
14         div.innerHTML = `
15             <h2>${p.nombre}</h2>
16             <p>Precio: ${p.precio}</p>
17             <button onclick="agregarCarrito('${p.nombre}')">Agregar
               al carrito</button>
18         `;
19         contenedor.appendChild(div);
20     });
21 }
22
23 async function agregarCarrito(nombre) {
24     let respuesta = await fetch("../functions/addCarrito.php", {
25         method: "POST",
26         headers: {"Content-Type": "application/json"},
27         body: JSON.stringify({ producto: nombre })
28     });
29
30     let carrito = await respuesta.json();
31     let lista = document.getElementById("carrito");
32     lista.innerHTML = "";
33     carrito.forEach(item => {
34         let li = document.createElement("li");
35         li.textContent = item;
36         lista.appendChild(li);
37     });
38 }
39
40 cargarProductos();
```

Ventajas:

Escalabilidad automática: La nube ajusta los recursos automáticamente según la carga (no tienes que preocuparte por servidores).

Coste por uso: Solo pagas cuando tu función se ejecuta, no por tener servidores encendidos todo el tiempo.

Menor mantenimiento: No gestionas servidores ni infraestructura: el proveedor cloud se encarga.

Despliegue rápido: Puedes crear y publicar funciones de manera independiente, lo que acelera el desarrollo.

Alta disponibilidad: Los proveedores (AWS, Azure, GCP) ofrecen redundancia y tolerancia a fallos integrados.

Ideal para cargas variables: Perfecto para sistemas con picos de tráfico (ej: campañas, ventas flash, apps de eventos).

Desventajas:

Latencia de arranque en frío: La primera vez que se ejecuta una función puede tardar más (cold start).

Limitaciones de ejecución: Tiempo máximo de ejecución (ej: 5?15 min según el proveedor).

No apto para procesos de larga duración.

Complejidad en depuración y pruebas: Más difícil hacer debugging y monitoreo en comparación con servidores tradicionales.

Dependencia del proveedor (vendor lock-in): Migrar funciones entre distintos proveedores puede ser complicado.

Gestión de estado complicada: Las funciones son stateless, por lo que el estado (carritos, sesiones, colas) debe manejarse con servicios externos (BD, cachés, colas de mensajes).

Costes ocultos en gran escala: Aunque barato al inicio, si el tráfico es muy alto, puede volverse más caro que servidores dedicados

1.4.8. SPA (Single Page Application)

Una SPA carga una sola página HTML y luego renderiza dinámicamente el contenido con JavaScript.

No se recarga toda la página, sino que el frontend se comunica con un backend/API REST para obtener datos y actualiza la vista con AJAX / Fetch / frameworks JS.

```
/mi_tienda_spa
├── frontend
│   ├── index.html
│   ├── style.css
│   └── app.js
└── api
    ├── db.php
    └── productos.php
```

Figura 1.7: Directorio arquitectura SPA (simulación).

```
1
2 // api/db.php
3
4 <?php
5 class Database {
6     private $host = "localhost";
7     private $user = "root";
8     private $pass = "usbw";
9     private $db   = "tienda";
```

```
10     public $conexion;
11
12     public function __construct() {
13         $this->conexion = new mysqli($this->host, $this->user,
14                                     $this->pass, $this->db);
15         if ($this->conexion->connect_error) {
16             die("Error de conexión: " .
17                 $this->conexion->connect_error);
18         }
19     }
20 }
21 ?>
```

```
1
2 // api/productos.php
3
4 // API que devuelve productos:
5
6 <?php
7 header("Content-Type: application/json");
8 require_once("db.php");
9
10 $db = new Database();
11 $sql = "SELECT * FROM productos";
12 $resultado = $db->conexion->query($sql);
13
14 $productos = [];
15 while ($fila = $resultado->fetch_assoc()) {
16     $productos[] = $fila;
17 }
18
19 echo json_encode($productos);
20 ?>
```

```
1
2 // frontend/index.html. La SPA vive en una sola página:
3
4
5 <!DOCTYPE html>
6 <html lang="es">
7 <head>
8     <meta charset="UTF-8">
9     <title>Tienda SPA</title>
10    <link rel="stylesheet" href="style.css">
11 </head>
12 <body>
13     <h1>Tienda con Arquitectura SPA</h1>
14     <nav>
15         <button onclick="mostrarSeccion('productos')">Productos</button>
16         <button onclick="mostrarSeccion('carrito')">Carrito</button>
17     </nav>
18
19     <div id="vista"></div>
20
21     <script src="app.js"></script>
22 </body>
23 </html>
```

```
1
2 // frontend/app.js
3
4 // Toda la lógica de navegación y renderizado ocurre aquí (SPA):
5
6
7 let carrito = [];
8
9 async function cargarProductos() {
10     let respuesta = await fetch("../api/productos.php");
11     let productos = await respuesta.json();
12
13     let html = "<h2>Productos</h2><div>";
14     productos.forEach(p => {
15         html += '
16             <div class="producto">
17                 <h3>${p.nombre}</h3>
18                 <p>Precio: ${p.precio}</p>
19                 <button onclick="agregarCarrito('${p.nombre}')">Agregar
                    al carrito</button>
20             </div>
21         ';
22     });
23     html += "</div>";
24     document.getElementById("vista").innerHTML = html;
25 }
26
27 function mostrarCarrito() {
28     let html = "<h2>Carrito</h2><ul>";
29     carrito.forEach(item => {
30         html += '<li>${item}</li>';
31     });
32     html += "</ul>";
33     document.getElementById("vista").innerHTML = html;
34 }
35
36 function mostrarSeccion(seccion) {
37     if (seccion === "productos") {
38         cargarProductos();
39     } else if (seccion === "carrito") {
40         mostrarCarrito();
41     }
42 }
43
44 function agregarCarrito(nombre) {
45     carrito.push(nombre);
46     alert(nombre + " agregado al carrito");
47 }
48
49 mostrarSeccion("productos");
```

Ventajas: Experiencia de usuario fluida: No se recarga toda la página, solo se actualizan partes necesarias ? sensación similar a una app de escritorio.

Rapidez tras la primera carga: Una vez descargado el frontend, las interacciones son rápidas gracias a AJAX / APIs REST / GraphQL.

Reutilización del backend: El frontend se conecta a APIs ? el mismo backend puede servir a web, móvil u otros clientes.

Desarrollo modular: Separación clara entre frontend (Vue, React, Angular) y backend (Node, PHP, Python, etc.).

Posibilidad de trabajar offline: Con Service Workers y almacenamiento local, una SPA puede funcionar parcialmente sin internet.

Desventajas:

SEO complicado: Al renderizarse en el navegador, los buscadores tienen más dificultad para indexar el contenido (aunque existen soluciones como SSR e hidratación).

Mayor tiempo de carga inicial: La primera visita puede ser más lenta porque debe descargarse toda la app (JS, CSS, etc.).

Complejidad en seguridad: Se expone más lógica en el frontend y depende de tokens para autenticación (JWT, OAuth, etc.).

Gestión de estado: Requiere librerías y patrones adicionales (Redux, Vuex, Context API) para manejar datos compartidos.

Dependencia de JavaScript: Si el usuario tiene JS deshabilitado (raro, pero posible), la app no funciona.

Mantenimiento más complejo en proyectos grandes: La app puede crecer mucho en el frontend y requerir una buena organización de módulos, rutas y servicios.

.